Threads, Multithreading Models & Thread Scheduling

Topics:

- Thread Concept
- Process vs Thread
- Multithreading
- Benefits of Multithreading
- Multithreading Models
- Thread Scheduling

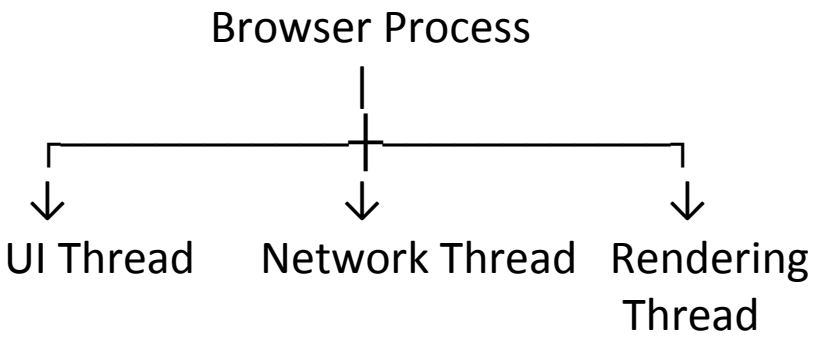
Why Do We Need Threads?

Imagine a web browser.
At the same time it may:

- Display the webpage
- Download data
- Play audio/video
- Handle keyboard/mouse input
- Run JavaScript
- Communicate with the network

If everything had to be handled by one execution path, the application could become less responsive.

Solution:
Multiple Threads



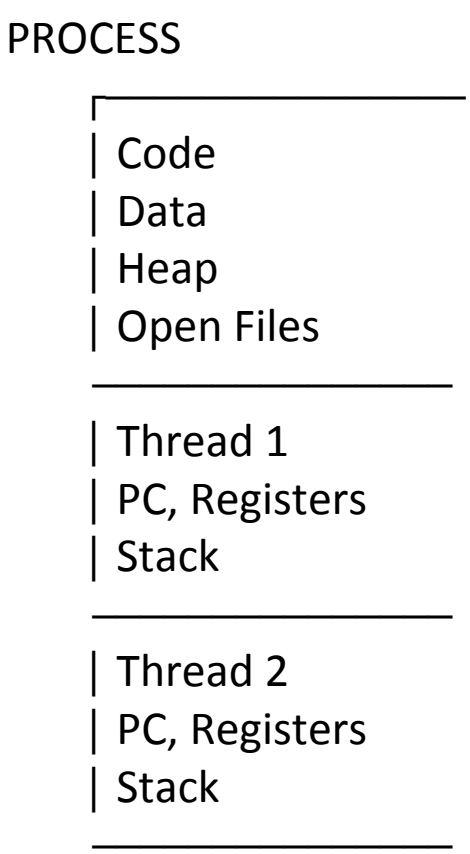
What is a Thread?

Definition
A thread is the smallest unit of CPU execution within a process.
A thread has its own:

- Program Counter
- CPU Registers
- Stack
- Thread State

But threads belonging to the same process share:

- Code
- Data
- Heap
- Open files/resources



Thread = Unit of execution
Process = Resource ownership + one or more threads

Process vs Thread

Process	Thread
Program in execution	Unit of execution within a process
Has its own address space	Shares address space with threads of same process
More expensive to create	Less expensive to create
Process switching is relatively expensive	Thread switching is generally cheaper
Processes are more isolated	Threads share resources
Communication requires IPC mechanisms	Communication can occur through shared data

Real life example

Department = Process

Faculty members = Threads

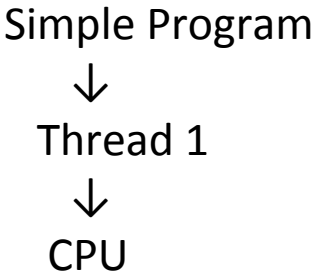
Common:
- Department office
- Files/resources
- Infrastructure

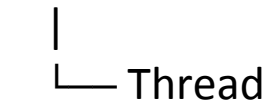
Individual:
- Each faculty member has their own work

“Threads work together inside the same process and can access common resources. That is why communication between threads is generally easier, but it also creates synchronization challenges.”

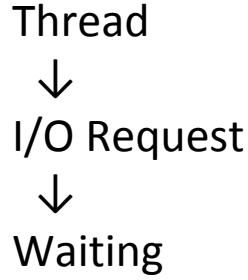
Single-Threaded Process

A process containing only one thread is called a:
Single-threaded process





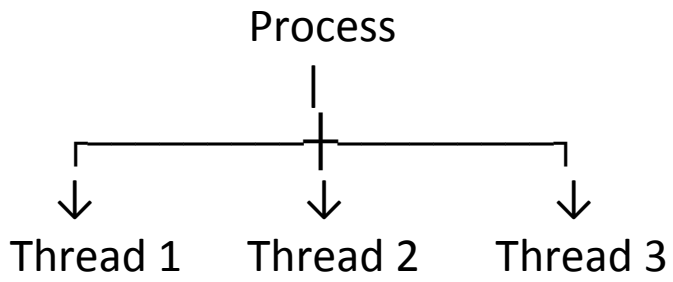
Problem
If the thread waits for an I/O operation:



The process may not be able to perform useful work during that period.

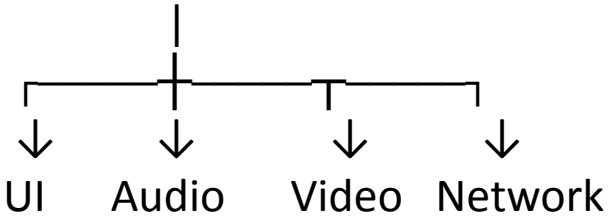
Multithreaded Process

A process containing multiple threads is called a:
Multithreaded Process



For example, a media player:

Media Player Process



Each thread performs a particular task.

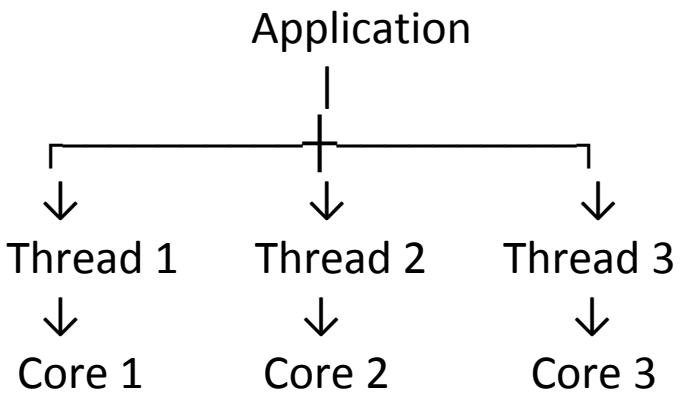
Benefits of Multithreading

1. Responsiveness
One thread can continue while another waits.
2. Resource Sharing
Threads share the resources of their process.
3. Economy
Creating and managing threads generally requires fewer resources than creating separate processes.
4. Scalability
Threads can execute in parallel on multicore processors.
5. Better CPU Utilization
CPU can execute another ready thread while one thread waits for I/O.
Easy Memory Trick
R-R-E-S

- **R**esponsiveness
- **R**esource sharing
- **E**conomy
- **S**calability

Multicore and Multithreading

Suppose a processor has:
4 CPU cores
A multithreaded application may have:



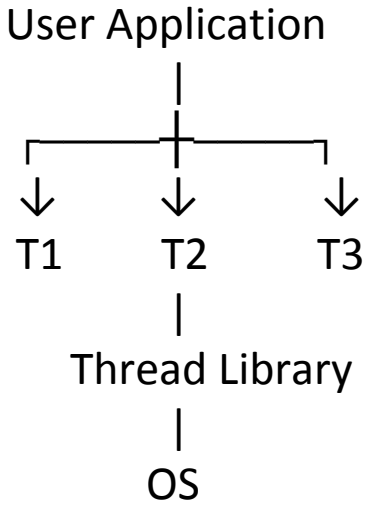
This can allow genuine parallel execution when the OS and hardware support it.
Important
Concurrency ≠ Parallelism

- **Concurrency:** Multiple tasks make progress during overlapping time periods.
- **Parallelism:** Multiple tasks actually execute simultaneously on different processing units.

User-Level Threads

User-Level Threads

Threads are managed by a **user-level thread library**, without requiring the OS kernel to manage each thread individually.



Advantages

- Fast thread management
- Thread creation can be efficient
- Kernel involvement can be reduced

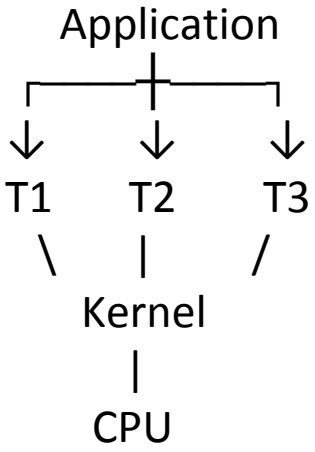
Disadvantage

If one thread makes a blocking system call, depending on the implementation, the entire process may be blocked.

Kernel-Level Threads

Kernel-Level Threads

The operating system kernel directly manages the threads.



Advantages

- Kernel can schedule individual threads.
- If one thread blocks, another thread of the same process may continue.
- Better support for multicore systems.

Disadvantage

Kernel-level thread management has more overhead than purely user-level management.

User-Level vs Kernel-Level Threads

Feature	User-Level	Kernel-Level
Managed by	Thread library	OS Kernel
Kernel awareness	Usually no direct awareness of each user thread	Yes
Management	Generally faster	More overhead
Blocking issue	Can affect entire process depending on model	Other threads can continue
Multicore support	Depends on mapping	Strong support
Scheduling	User-level library may schedule	Kernel schedules

Multithreading Models

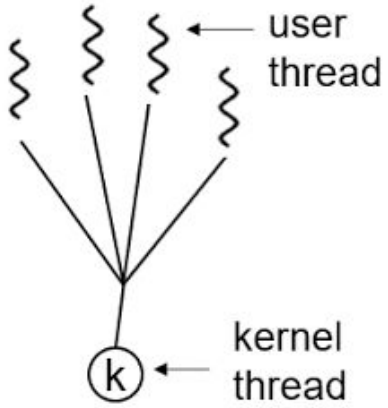
How are user threads mapped to kernel threads?

There are three major models:

1. **Many-to-One**
2. **One-to-One**
3. **Many-to-Many**

Many-to-One Model

Many User Threads → One Kernel Thread



Many-to-one
thread model

Advantages

- Simple
- Thread management can be efficient

Disadvantages

- If one thread blocks, the process may block.
- Cannot achieve true parallelism on multiple CPU cores because only one kernel thread is available.

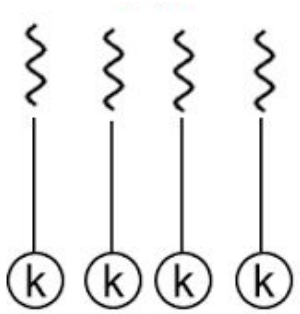
Remember:

Many → One

One-to-One Model

One User Thread → One Kernel Thread

T1 ——— K1
T2 ——— K2
T3 ——— K3
T4 ——— K4



One-to-one thread model

Advantages

- Better concurrency
- One blocked thread does not necessarily block other threads.
- Can take advantage of multiple CPU cores.

Disadvantage

Creating a large number of kernel threads can increase system overhead.

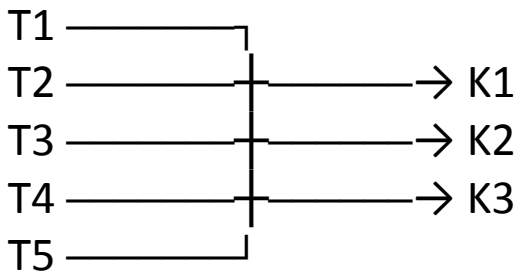
Remember:

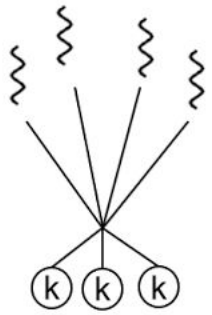
One → One

Many-to-Many Model

Many User Threads → Many Kernel Threads

User Threads Kernel Threads





Many-to-many thread model

Multiple user threads can be mapped onto a smaller or equal number of kernel threads.

Advantages

- Flexibility
- Supports concurrency
- Kernel threads can execute on multiple processors.

Main Idea

Many user threads are multiplexed over a pool of kernel threads.

Comparison of Multithreading Models

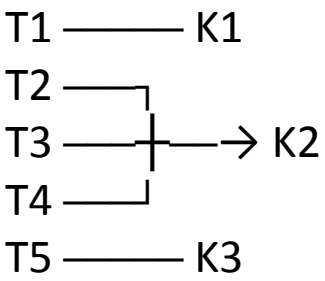
Model	Mapping	Parallelism	Main Issue
Many-to-One	Many → One	No true parallelism	Blocking problem
One-to-One	One → One	Yes	Higher overhead
Many-to-Many	Many → Many	Yes	More complex

Easy Memory Trick

Many → One = Simple
One → One = Parallel but expensive
Many → Many = Flexible

Two-Level Model

Some operating-system texts also discuss the **Two-Level Model**.
It combines features of Many-to-Many with the ability to bind particular user threads to specific kernel threads.



Key Idea
Most user threads can be multiplexed, while selected threads can be permanently bound to kernel threads.

Thread Libraries

A **thread library** provides functions for creating and managing threads.

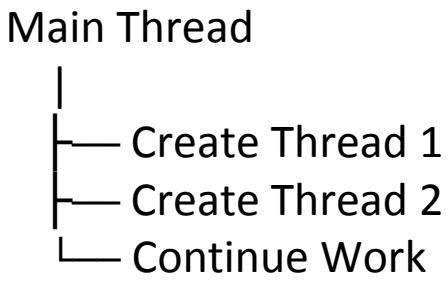
Examples:

- **POSIX Pthreads**
- **Windows Threads**
- **Java Threads**

Typical operations:

Create Thread
↓
Execute
↓
Wait / Join
↓
Terminate

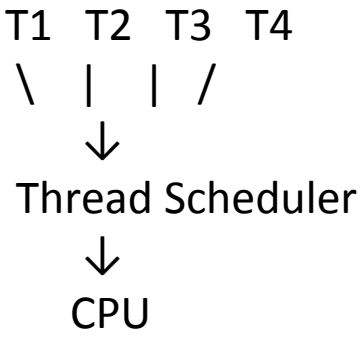
Conceptual Example



Thread Scheduling

Thread Scheduling
The operating system or thread library determines:
Which ready thread should execute next?
Example:

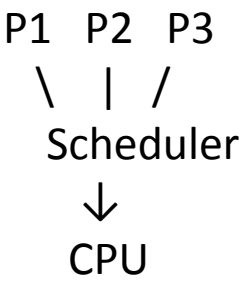
Ready Threads



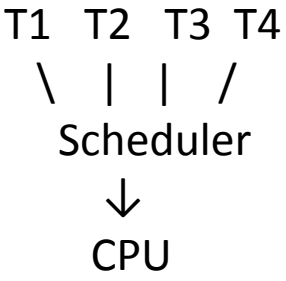
The scheduling decision depends on the threading implementation and operating system.

Process Scheduling vs Thread Scheduling

Process Scheduling
Scheduler selects among processes.



Thread Scheduling
Scheduler selects among executable threads.



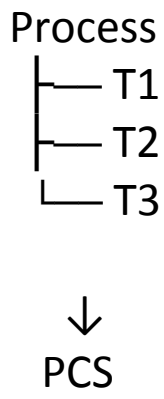
Important Point
In a multithreaded system, the CPU ultimately executes **threads**, not an abstract process as a whole.

Thread Scheduling Scope

Two important concepts:

1. Process-Contention Scope (PCS)

Competition among threads **within the same process**.



2. System-Contention Scope (SCS)

Threads compete with other threads in the **entire system** for CPU resources.

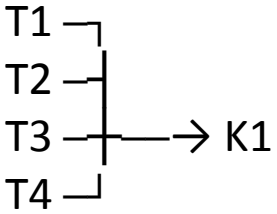
P1: T1 T2
P2: T3 T4
P3: T5 T6

↓
System
↓
CPU

PCS → within a process
SCS → system-wide

Activity: Identify the Thread Model

Case 1

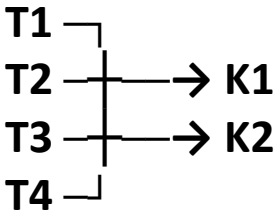


Question: Which model?

Case 2

T1 → K1
T2 → K2
T3 → K3

Case 3



Mini Problem: Process vs Threads

Consider a Music Player

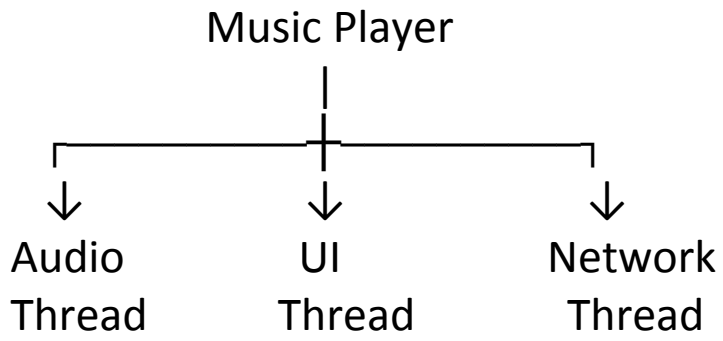
It performs:

1. Playing music
2. Updating user interface
3. Downloading album artwork
4. Reading keyboard/mouse input

Should we create four separate processes or multiple threads inside one process?

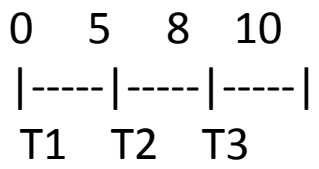
Expected Discussion

Multiple threads can be used when these activities belong to the same application and need to share resources.



5-Minute Numerical/Conceptual Activity

Thread	CPU Burst
T1	5 ms
T2	3 ms
T3	2 ms



T1

CT = 5

TAT = 5

WT = 0

T2

CT = 8

TAT = 8

WT = 5

T3

CT = 10

TAT = 10

WT = 8

Average Waiting Time

$$AWT = \frac{0 + 5 + 8}{3} AWT = 4.33\ ms$$

Important Terms

Remember these terms:

Thread

→ Smallest unit of CPU execution.

Multithreading

→ Multiple threads within a process.

User-Level Thread

→ Managed primarily by a user-level thread library.

Kernel-Level Thread

→ Managed by the operating system kernel.

Many-to-One

→ Many user threads → One kernel thread.

One-to-One

→ One user thread → One kernel thread.

Many-to-Many

→ Many user threads → Many kernel threads.

PCS

→ Process-Contention Scope.

SCS

→ System-Contention Scope.

MCQ Quiz

Q1. A thread is:
A. A complete operating system B. The smallest unit of CPU execution
C. A type of memory D. A file

Q2. Which is generally private to each thread?
A. Code B. Heap C. Program Counter D. Data section

Q3. In the Many-to-One model:
A. One user thread maps to one kernel thread
B. Many user threads map to one kernel thread
C. One kernel thread maps to many processes
D. Many processes map to one user thread

Q4. Which model provides one kernel thread for each user thread?
A. Many-to-One B. Many-to-Many C. One-to-One D. Two-to-One

Q5. PCS stands for:
A. Process Control System B. Process-Contention Scope
C. Program Control Scheduler D. Process Communication Service

B
C
B
C
B